



SAPIENZA
UNIVERSITÀ DI ROMA

DIPARTIMENTO DI INFORMATICA

Simulazione Crescita Cristallina

PROGRAMMAZIONE DI SISTEMI EMBEDDED E MULTICORE

Professore:

Salvatore Pontarelli

Studenti:

Francesco Di Monaco (1988475)

Vincenzo Malanga (2024764)

Academic Year 2023/2024

Contents

1	Descrizione del progetto	2
2	Il nostro approccio	2
3	Architettura	2
4	Algoritmo	3
4.1	Popolare le celle della matrice con linked-list vuote	3
4.2	Popolare le linked-list con le particelle	3
4.3	Ordinare le linked-list	3
4.4	Verificare eventuali collisioni	4
4.5	Muovere le particelle	4

1 Descrizione del progetto

Diffusion-limited aggregation (DLA) è un processo di formazione di cristalli nel quale le particelle si muovono in uno spazio 2D con moto browniano (cioè in modo casuale) e si combinano tra loro quando si toccano. DLA può essere simulato utilizzando una griglia 2D in cui ogni cella può essere occupata da uno o più particelle in movimento. Una particella diventa parte di un cristallo (e si ferma) quando si trova in prossimità di un cristallo già formato. I parametri di base della simulazione sono la dimensione della griglia 2D, il numero iniziale di particelle, il numero di iterazioni e il "seme" cristallino iniziale. Per la nostra simulazione abbiamo deciso di utilizzare i framework OpenMP e CUDA.

2 Il nostro approccio

L'obiettivo che ci siamo preposti consiste nel permettere alle particelle di muoversi liberamente in qualsiasi punto dello spazio, indipendentemente dalla griglia menzionata nella consegna. Per questo motivo, nella nostra architettura, le particelle occupano una zona di memoria separata da quella della griglia. In quest'ultima, infatti, sono memorizzati solo i puntatori alle particelle.

Ci teniamo a sottolineare che la griglia è utilizzata esclusivamente come supporto per l'algoritmo e non è necessaria per l'esistenza delle particelle stesse. Per limitare notevolmente il numero di controlli necessari, abbiamo adottato approcci che includono memoization, caching e sfruttamento della località delle particelle.

3 Architettura

Il nostro algoritmo è implementato tramite una matrice, le cui celle contengono una linked-list ciascuna. Così facendo evitiamo la pre-allocazione preventiva di spazio per ogni cella, impiegando invece gli struct delle particelle come nodi delle linked-list. Ad ogni particella nello spazio corrisponde una cella della griglia, a seconda delle coordinate. La griglia viene popolata e svuotata a ogni iterazione. Questa architettura ci consente di sfruttare la località delle particelle ed evitare controlli superflui.

4 Algoritmo

Il nostro algoritmo prevede i seguenti passaggi:

1. Popolare le celle della matrice con linked-list vuote.
2. Popolare le linked-list con le particelle.
3. Ordinare le linked-list.
4. Verificare eventuali collisioni.
5. Muovere le particelle.
6. Ricominciare dal punto 1.

4.1 Popolare le celle della matrice con linked-list vuote

In questa fase inizializziamo o resettiamo le teste delle linked-list, preparando così la matrice per il passaggio successivo: l'inserimento delle particelle. L'utilizzo delle linked-list è una scelta obbligata. Se avessimo optato per array di dimensione fissa, avremmo dovuto allocare molta memoria e sprecarne altrettanta. Un'alternativa sarebbe stata l'uso di ArrayList in ogni cella, ma questo avrebbe comunque comportato sprechi di memoria e difficoltà nell'implementare una struttura come l'ArrayList in CUDA.

4.2 Popolare le linked-list con le particelle

Iteriamo sull'array delle particelle e per ognuna di esse inseriamo il puntatore nella cella corrispondente alle sue coordinate. Questo passaggio richiede l'uso di lock o meccanismi di sincronizzazione per consentire l'inserimento nelle liste, garantendo così la thread-safety. In questa fase dell'algoritmo, l'implementazione può variare leggermente a seconda del framework utilizzato.

Ad esempio, in CUDA abbiamo incontrato difficoltà nell'implementazione di sezioni critiche con i lock, rendendo necessario l'utilizzo del check-and-set. In OpenMP, invece, l'implementazione è stata possibile solamente mediante l'utilizzo di lock mutex.

4.3 Ordinare le linked-list

Ogni linked-list viene ordinata mediante un algoritmo di ordinamento. Questo passaggio è essenziale per rendere lineare la complessità del passaggio successivo.

Per comprendere meglio l'importanza di questa operazione, consideriamo il caso in cui non venisse eseguita: il passaggio successivo richiede la verifica delle collisioni tra coppie di celle. Se le particelle all'interno di queste celle non fossero ordinate, saremmo

comunque costretti a utilizzare un algoritmo che renda questi controlli più efficienti, probabilmente attraverso processi simili all'ordinamento. Ordinare le celle in anticipo ci permette di eseguire questi processi una sola volta per ogni cella, agendo come una sorta di memoizzazione.

4.4 Verificare eventuali collisioni

Iteriamo sulle celle della matrice e, per ciascuna di esse, controlliamo le collisioni con le particelle presenti nelle celle adiacenti. L'algoritmo sviluppato per la verifica delle collisioni prende in input due celle alla volta, quindi iteriamo sulla matrice e per ogni cella eseguiamo l'algoritmo n volte (sulle celle circostanti), dove n dipende dal tipo di cella*. Grazie al fatto che le linked-list presenti nelle celle sono ordinate, l'algoritmo può operare con complessità lineare, sfruttando la pseudo-memoizzazione del passaggio precedente.

* Per evitare controlli ridondanti, abbiamo sviluppato dei pattern che, a seconda del tipo di cella, determinano quali celle adiacenti devono essere controllate, evitando di verificare tutte e nove le celle circostanti.

4.5 Muovere le particelle

Iteriamo sull'array delle particelle ed in ogni struct cambiamo le coordinate di 1 pixel. La direzione è data da un random number generator.